

Impact of Prompt Formatting on Instruction Adherence in Large Language Models

Batuhan Yılmaz
Dept. of Computer Engineering
Akdeniz University
Antalya, Turkey
20220808068

Emre Kemal Aksel
Dept. of Computer Engineering
Akdeniz University
Antalya, Turkey
20220808065

Abstract—This report investigates the impact of structural prompt formatting (Plain Text, Markdown, XML, JSON, and YAML) on the instruction adherence capabilities of Large Language Models (LLMs). Instruction adherence was evaluated using two primary benchmarks: IFEval and IFBench. Due to excessive computational overhead, LLM-as-a-judge frameworks such as WildBench were bypassed in favor of deterministic, programmatically verifiable evaluation. We detail our extensive model selection journey, migrating from Qwen 3.5 9B and Gemini 2.5 Flash to our final model, DeepSeek v3.1, while addressing inference bottlenecks such as infinite reasoning loops, degenerative repetition, and context window overflows. Our findings indicate that for shorter tasks with strict verifiable rules (IFEval), XML formatting yields the highest constraint adherence (89.28% strict accuracy). Conversely, for longer and more complex out-of-distribution instructions (IFBench), the distinction between structural formats diminishes, with JSON marginally outperforming the others (50.67% strict accuracy). These results highlight the complex correlation between task context length, constraint strictness, and optimal prompt structuring.

Index Terms—Large Language Models, Prompt Engineering, Instruction Following, IFEval, IFBench

I. INTRODUCTION

As Large Language Models (LLMs) are increasingly integrated into agentic workflows and complex applications, developers rely heavily on structural prompt syntax (e.g., XML, JSON, Markdown) to guide the model’s behavior. Despite this reliance, there is limited empirical consensus on which formatting style objectively yields the highest level of constraint adherence.

This project addresses this gap by systematically evaluating how prompt formatting impacts an LLM’s ability to follow complex, multi-step instructions. We built our evaluation pipeline upon the foundational Instruction Following Evaluation (IFEval) benchmark [1], which tests strict verifiable rules (e.g., specific word counts or forbidden letters), and IFBench [2], which introduces harder, out-of-distribution (OOD) constraints. We synthesized five exact format permutations (Plain Text, Markdown, XML, JSON, YAML) for each baseline prompt to strictly isolate the syntax variable. Our research chronicles not only the final quantitative outcomes but also the significant computational and architectural hurdles involved in generating responses using modern reasoning models.

Project Repository: <https://github.com/BatY0/LLMTermProject>

II. RELATED WORK

Evaluating LLM instruction adherence requires reliable, scalable benchmarks. IFEval [1] provides a highly deterministic framework by utilizing short Python scripts to programmatically verify whether an LLM followed constraints, eliminating the subjectivity of human evaluation. IFBench [2] extends this paradigm by introducing longer, more challenging out-of-domain constraints to test a model’s true generalization capabilities.

In our initial research phase, we heavily considered WildBench [3], which benchmarks models using challenging tasks derived from real-world conversations. However, WildBench utilizes an LLM-as-a-judge approach. Because our experimental design required testing five distinct format permutations for every single prompt, using an LLM-as-a-judge would require deploying two separate agents for a single evaluation approach. We determined that multiplying this dual-agent architecture by five formats per prompt was computationally and financially infeasible. Consequently, WildBench was excluded in favor of the deterministic scripts offered by IFEval and IFBench.

III. METHODOLOGY

A. Data Modification and Restructuring

To isolate the formatting variable, every prompt in the selected benchmarks needed to be meticulously converted into five distinct variations: Plain, Markdown, XML, JSON, and YAML. Because automated scripts proved insufficient for cleanly separating core tasks from explicit constraints in complex prompts, we altered the data sequentially. We developed a custom HTML-based interface to manually iterate through the dataset item-by-item. This interface allowed us to restructure the raw prompts into a task/rule-oriented representation, clearly demarcating the task content from the rule content, and subsequently outputting a newly structured JSON dataset containing the five formatting variants.

B. Model Selection and Inference Challenges

The process of generating text outputs across all variants involved a highly iterative model selection journey to overcome computational bottlenecks:

- **Attempt 1: Qwen 3.5 9B (Google Colab).** We initially attempted to run inference locally using Qwen 3.5 9B on

a free tier Google Colab instance. Despite optimizations, the execution speed (estimated alongside T5) indicated it would take approximately 51 hours to process a single dataset. Given Colab’s 4-hour session limit, this approach was abandoned.

- **Attempt 2: Qwen (OpenRouter API).** To bypass local hardware limits, we transitioned to the OpenRouter API. However, using the Qwen model via the API introduced severe bottlenecks associated with its “thinking” (Chain-of-Thought) process. Processing 32k-token context inputs took an excessive amount of time, restricting our throughput to merely 6 prompts per hour. More critically, this extended reasoning frequently led to context overflow. Once the context window was breached, the model would abruptly restart its generation from zero. This caused highly inconsistent behavior and forced us into an endless loop of retrying the exact same prompts repeatedly without success.
- **Attempt 3: Gemini 2.5 Flash.** We switched to Gemini 2.5 Flash to increase inference speed. While the throughput vastly improved, the model suffered from qualitative logic issues. It exhibited degenerative repetition, caught itself in loops, and ultimately overflowed its own context length limit.
- **Final Model: DeepSeek v3.1.** To balance reasoning capability with execution stability, we finalized our inference pipeline using DeepSeek v3.1. To mitigate the risk of the model entering an infinite reasoning loop, consuming too many tokens, or failing to finish within a practical timeframe, we implemented a strict token limit of 4096. This successfully resolved the repetition issues and stabilized our dataset generation.

Furthermore, to guarantee deterministic behavior and fairly compare the structural formatting variants, the model’s generation temperature was strictly set to 0 across all inference runs. This ensured that the model produced the exact same logical pathways for identical prompts, isolating the prompt structure as the sole independent variable.

C. Evaluation Pipeline

Once the model outputs were aggregated via OpenRouter, we formatted the responses into benchmark-compatible JSONL files. We modified the standard IFEval grading scripts so they could automatically evaluate and aggregate both strict and loose accuracy scores across our five custom syntax groupings.

IV. RESULTS AND DISCUSSION

A. IFEval Results

IFEval features relatively short tasks bounded by highly strict formatting rules. As shown in Table I, the XML format achieved the highest performance by a notable margin, reaching an 89.28% Strict Accuracy score. We hypothesize that in shorter contexts with rigid constraints, XML provides the clearest structural boundaries, allowing the LLM to effectively isolate the rules from the task content without hallucinating.

TABLE I
IFEVAL PROMPT-LEVEL ACCURACY

Rank	Format	Strict (%)	Loose (%)
1	XML	89.28	91.87
2	YAML	85.95	87.98
3	Markdown	85.76	88.90
4	JSON	85.21	88.53
5	Plain	85.02	88.72

B. IFBench Results

IFBench is inherently more difficult, comprising longer tasks with fewer explicit formatting instructions but highly complex out-of-domain constraints. As observed in Table II, overall performance dropped drastically across all formats, reflecting the benchmark’s difficulty. Here, JSON emerged as the highest-performing format, achieving a 50.67% Strict Accuracy.

TABLE II
IFBENCH PROMPT-LEVEL ACCURACY

Rank	Format	Strict (%)	Loose (%)
1	JSON	50.67	52.67
2	Plain	48.67	51.33
3	Markdown	47.00	48.67
4	XML	46.67	49.33
4	YAML	46.67	49.33

C. Impact of Task Length and Format Structure

A critical observation from our results is the behavioral shift between the two benchmarks. In IFEval, the distinct syntax of XML drastically improved adherence. However, in IFBench, the performance gap between the best format (JSON) and the worst (XML/YAML) narrowed to just 4%.

This suggests that in longer tasks with fewer explicit instructions, the divide between different formatting paradigms diminishes. The structural wrapper plays a less significant role when the LLM is forced to manage larger cognitive loads over extended context windows. Furthermore, the 4096-token limitation we enforced on DeepSeek v3.1—while necessary to prevent repetition and endless reasoning—may have restricted the model’s internal Chain-of-Thought capacity, contributing to the lower overall scores on the more complex IFBench dataset compared to IFEval.

V. CONCLUSION

This project systematically evaluated how five distinct prompt formats impact LLM instruction adherence. Through extensive iterative testing, we concluded that structural prompt formatting heavily influences model performance on short, rule-strict tasks, with XML yielding the highest fidelity. However, as task length and complexity increase, the performance distinction between structural wrappers shrinks. Additionally, deploying modern reasoning models requires a careful

balance; while expansive context windows allow for deep reasoning, capping token limits is often a necessary tradeoff to prevent inference bottlenecks and degenerative repetition. Ensuring a temperature of 0 provided a solid baseline, verifying that these observed format-driven differences were consistent and measurable rather than byproducts of stochastic generation.

REFERENCES

- [1] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-Following Evaluation for Large Language Models," *arXiv preprint arXiv:2311.07911*, 2023.
- [2] AllenAI / V. Pyatkin et al., "Generalizing Verifiable Instruction Following" (IFBench), *arXiv preprint*, 2025. [Online]. Available: <https://arxiv.org/pdf/2512.09663>
- [3] B. Y. Lin, Y. Deng, K. Chandu, F. Brahman, A. Ravichander, V. Pyatkin, N. Dziri, R. Le Bras, and Y. Choi, "WildBench: Benchmarking LLMs with Challenging Tasks from Real Users in the Wild," *arXiv preprint arXiv:2406.04770*, 2024.